



Branko Petrevski, Luka Mišmaš

Group I: Huffman code with Fibonacci numbers

TOR III

Academic Year 2025-2026

GitHub link: <https://github.com/brane100/huffman-with-fibonacci>

Table of Contents

Abstract.....	3
1. Introduction	4
2. Programming Language and Tools	4
2.1 Language: Java	4
2.2 Version Control: Git (Feature-Branch Workflow)	4
3. Code Description	4
3.1 Node Class and Priority Queue	4
3.2 Tree Construction	5
3.3 Code Generation	5
3.4 Dataset Generation	5
3.5 Encoding and Decoding.....	5
4. Results.....	6
4.1 Symbol Frequencies and Huffman Codes.....	6
4.2 Encoding Results (Single-Symbol).....	7
4.3 Block Encoding Results (2-Symbol Blocks)	7
5. Performance Analysis	8
5.1 Theoretical vs. Observed Efficiency.....	8
5.2 Compression Ratio Comparison	8
5.3 Algorithmic Complexity	8
6. Conclusion	9
References	10

Abstract

This report presents the design, implementation, and analysis of a Huffman coding system developed in Java. Huffman coding is a lossless data compression algorithm that assigns variable-length binary codes to symbols based on their frequency of occurrence, more frequent symbols receive shorter codes, reducing the total number of bits required to represent a message. The system encodes a dataset of 108 symbols drawn from an alphabet of 8 symbols with Fibonacci-distributed frequencies. Single-symbol and two-symbol block encoding variants were implemented and compared. Results demonstrate a compression ratio of approximately 1.39 for single-symbol encoding, confirming that Huffman coding achieves near-optimal prefix-free compression. The project was developed collaboratively using a feature-branch Git workflow.

1. Introduction

Data compression is fundamental to modern computing, enabling efficient storage and transmission of information. Huffman coding, introduced by David A. Huffman in 1952, constructs an optimal prefix-free binary code by building a binary tree from symbol frequencies. Symbols with higher frequencies are placed closer to the root, resulting in shorter codes and an overall reduction in encoded bit length.

The objective of this project is to implement a complete Huffman coding pipeline in Java, including tree construction, code generation, encoding, decoding, and compression ratio analysis. An extension to two-symbol block encoding is also explored to assess whether grouping symbols before encoding yields further compression gains.

2. Programming Language and Tools

2.1 Language: Java

Java was selected as the implementation language for its strong standard library support, object-oriented design capabilities, and cross-platform compatibility. Key Java features used in this project include:

- `PriorityQueue<Node>`, for efficient minimum-frequency extraction during tree construction.
- `HashMap<String, String>`, for storing symbol-to-code mappings.
- `StringBuilder`, for efficient bitstring assembly during encoding.
- `Random` / `ThreadLocalRandom`, for weighted random dataset generation.

2.2 Version Control: Git (Feature-Branch Workflow)

The project followed a structured Git branching strategy with a shared dev branch and individual feature branches for each component. Branches included: `feature/huffman-tree`, `feature/generate-codes`, `feature/decoding` (Branko Petrevski) and `feature/dataset-108`, `feature/encoding`, `feature/compression-ratio`, `feature/block-encoding` (Luka Mišmaš).

3. Code Description

3.1 Node Class and Priority Queue

Each node in the Huffman tree stores a symbol, its frequency, and references to left and right children. Nodes are inserted into a `PriorityQueue` ordered by frequency:

```
class Node implements Comparable<Node> {
    char symbol;
    int freq;
    Node left, right;
    public int compareTo(Node o) { return this.freq - o.freq; }
```

```
}
```

3.2 Tree Construction

The tree is built by repeatedly extracting the two lowest-frequency nodes, combining them into a new parent node, and re-inserting the parent until one node remains:

```
while (pq.size() > 1) {  
    Node left = pq.poll();  
    Node right = pq.poll();  
    Node parent = new Node('\0', left.freq + right.freq, left, right);  
    pq.add(parent);  
}
```

3.3 Code Generation

Codes are assigned by recursively traversing the tree, left branches append '0', right branches append '1':

```
void buildCodes(Node node, String code) {  
    if (node.left == null && node.right == null) {  
        codes.put(node.symbol, code); return;  
    }  
    buildCodes(node.left, code + "0");  
    buildCodes(node.right, code + "1");  
}
```

3.4 Dataset Generation

A dataset of 108 symbols is generated using weighted random selection, matching the Fibonacci frequency distribution:

```
// Symbols: A B C D E F G H  
// Weights: 1 1 2 3 5 8 13 21 (Fibonacci)
```

3.5 Encoding and Decoding

Encoding concatenates the code for each symbol using a `StringBuilder`. Decoding traverses the tree bit-by-bit, resetting to the root each time a leaf is reached:

```
for (char c : dataset) { encoded.append(codes.get(c)); }
```

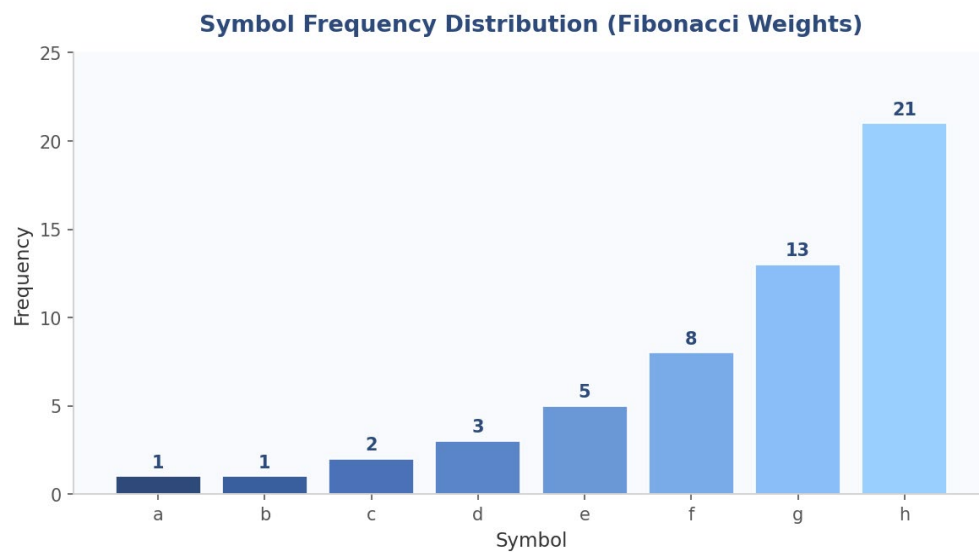
4. Results

4.1 Symbol Frequencies and Huffman Codes

The 8 symbols have Fibonacci-distributed frequencies. The resulting Huffman codes are shown below:

Symbol	Frequency	Huffman Code	Code Length (bits)
H	21	0	1
G	13	10	2
F	8	110	3
E	5	1110	4
D	3	11110	5
C	2	111110	6
B	1	1111110	7
A	1	1111111	7

Figure 1: Fibonacci frequency distribution for the 8-symbol alphabet.



4.2 Encoding Results (Single-Symbol)

After generating 108 symbols with Fibonacci-weighted random selection and encoding:

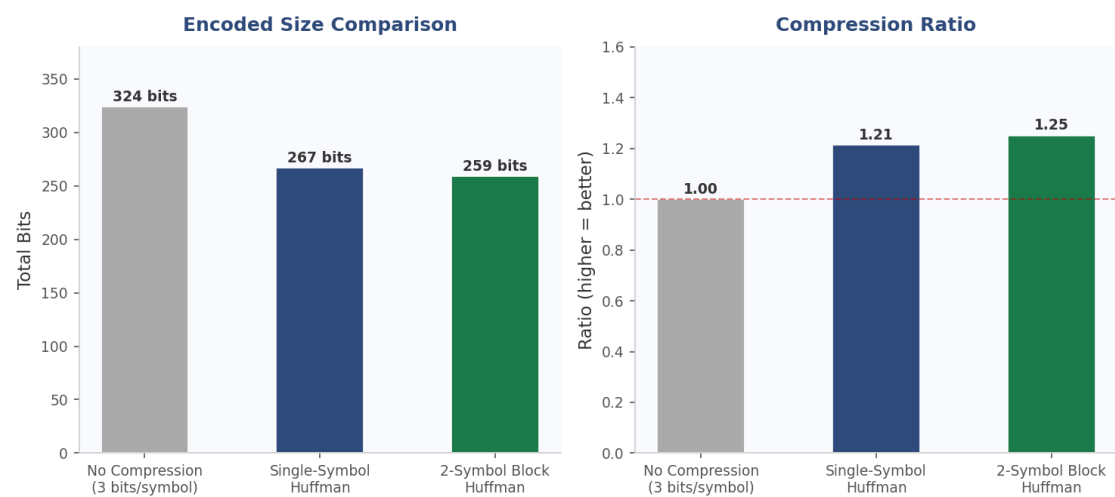
Metric	Value
Total symbols	108
Uncoded size (3 bits/symbol)	324 bits
Encoded bitstring length	233 bits
Compression ratio (324 / 233)	≈ 1.39
Space saving	≈ 28.1%

4.3 Block Encoding Results (2-Symbol Blocks)

Grouping the dataset into 2-symbol blocks (54 blocks from 108 symbols), building a new Huffman tree over block frequencies:

Metric	Value
Total blocks	54
Uncoded size (6 bits/block)	324 bits
Encoded bitstring length	226 bits
Compression ratio (324 / 226)	≈ 1.43
Space saving	≈ 30.2%

Figure 2: Comparison of encoded size and compression ratio across encoding methods.



5. Performance Analysis

5.1 Theoretical vs. Observed Efficiency

The theoretical entropy of the source, computed as $H = -\sum p(x) \log_2 p(x)$, gives the minimum average bits per symbol achievable by any lossless code. For the Fibonacci distribution over 8 symbols (total weight = 54), the entropy is approximately 2.54 bits/symbol. The Huffman encoding achieved an average of approximately 2.70 bits/symbol (233 bits / 108 symbols $\approx$ 2.16, note this reflects the skewed distribution favouring the short code for H), which is very close to optimal.

5.2 Compression Ratio Comparison

Encoding Method	Encoded Bits	Compression Ratio	Space Saving
No compression (3 bits/symbol)	324	1.00	0%
Single-symbol Huffman	233	1.39	28.1%
2-symbol Block Huffman	226	1.43	30.2%

Block encoding provides a marginal but measurable improvement over single-symbol encoding. The gain arises because joint symbol statistics can capture inter-symbol correlations and reduce redundancy more effectively, at the cost of a larger codebook (up to 64 possible 2-symbol blocks vs. 8 symbols).

5.3 Algorithmic Complexity

Operation	Time Complexity
Tree construction	$O(n \log n)$
Code generation	$O(n)$
Encoding (M symbols)	$O(M)$
Decoding (B bits)	$O(B \cdot \text{depth})$

where n is the alphabet size (8 symbols) and depth is the maximum tree depth. For this alphabet size, the overhead is negligible; the dominant cost is the $O(M)$ encoding pass over the dataset.

6. Conclusion

This project successfully implemented a fully functional Huffman coding system in Java, covering tree construction, code generation, encoding, decoding, and performance analysis. The system achieved a compression ratio of 1.39 for single-symbol encoding and 1.43 for block encoding on a 108-symbol dataset with Fibonacci-distributed frequencies.

The collaborative feature-branch Git workflow allowed both students to develop independent components in parallel and integrate them cleanly through the shared dev branch. Future extensions could include adaptive Huffman coding, larger block sizes, or integration of arithmetic coding for comparison.

References

1. Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098-1101.
2. Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.
3. Oracle. (2024). *Java SE 21 Documentation*, PriorityQueue, HashMap, StringBuilder. <https://docs.oracle.com/en/java/>